

Introduction to MSFconsole

To start interacting with the Metasploit Framework, we need to type `msfconsole` in the terminal of our choice. Many security-oriented distributions such as Parrot Security and Kali Linux come with `msfconsole` preinstalled. We can use several other options when launching the script as with any other command-line tool. These vary from graphical display switches/options to procedural ones.

Preparation

Upon launching the `msfconsole`, we are met with their coined splash art and the command line prompt, waiting for our first command.

Launching MSFconsole

Launching MSFconsole

```
dbcypth0n@htb[/htb]$ msfconsole
```

```

      `:oDfO:`
      ./ymM0dayMmy/.
      --dHJ5aGFyZGVyIQ==+-
      `:sm@~~Destroy.No.Data~~s:`
      -+h2~~Maintain.No.Persistence~~h+-
      `:odNo2~~Above.All.Else.Do.No.Harm~~Ndo:`
      ./etc/shadow.0days-Data'%200R%201=1--.No.0MN8'/.
      -++SecKCoin++e.Amd`      `.-://///hbove.913.ElsMNH+-
      ~~/ssh/id_rsa.Des-      `htN01UserWroteMe!-
      :dopeAW.No<nano>o      :is:TRiKC.sudo-.A:
      :we're.all.alike``      The.PFYroy.No.D7:
      :PLACEDRINKHERE!:      yxp_cmdshell.Ab0:
      :msf>exploit -j.      :Ns.BOB&ALICEes7:
      :---srwxrwx:-.``      `MS146.52.No.Per:
      :<script>.Ac816/      sENbove3101.404:
      :NT_AUTHORITY.Do      `T:/shSYSTEM-.N:
      :09.14.2011.raid      /STFU|wall.No.Pr:
      :hevnsntSurb025N.      dNVRG0ING2GIVUUP:
      :#OUTHOUSE- -s:      /corykennedyData:
      :$nmap -oS      SSo.6178306Ence:
      :Awsmda:      /shMTL#beats3o.No.:
      :Ring0:      `dDestRoyREXKC3ta/M:
      :23d:      sSETEC.ASTRONOMYist:
      /-      /yo- .ence.N:(){ :|: & };;
      `:Shall.We.Play.A.Game?tron/
      ``-ooy.if1ghtf0r+ehUser5`
      ..th3.H1V3.U2VjRFNN.jMh+.`
      `MjM~~WE.ARE.se~~MMjMs
      +~KANSAS.CITY's~-`
      J~HAKCERS~./.`
      .esc:wq!:`
      +++ATH`
      `

```

```

      =[ metasploit v6.1.9-dev      ]
+ -- --=[ 2169 exploits - 1149 auxiliary - 398 post      ]
+ -- --=[ 592 payloads - 45 encoders - 10 nops      ]
+ -- --=[ 9 evasion      ]

```

Metasploit tip: Use sessions -1 to interact with the last opened session

```
msf6 >
```

Alternatively, we can use the `-q` option, which does not display the banner.

Launching MSFconsole

```
dbcyph0n@htb[/htb]$ msfconsole -q  
msf6 >
```

To better look at all the available commands, we can type the `help` command. First things first, our tools need to be sharp. One of the first things we need to do is make sure the modules that compose the framework are up to date, and any new ones available to the public can be imported.

The old way would have been to run `msfupdate` in our OS terminal (outside `msfconsole`). However, the `apt` package manager can currently handle the update of modules and features effortlessly.

Installing MSF

Installing MSF

```
dbcyph0n@htb[/htb]$ sudo apt update && sudo apt install metasploit-framework  
  
<SNIP>  
  
(Reading database ... 414458 files and directories currently installed.)  
Preparing to unpack .../metasploit-framework_6.0.2-0parrot1_amd64.deb ...  
Unpacking metasploit-framework (6.0.2-0parrot1) over (5.0.88-0kali1) ...  
Setting up metasploit-framework (6.0.2-0parrot1) ...  
Processing triggers for man-db (2.9.1-1) ...  
Scanning application launchers  
Removing duplicate launchers from Debian  
Launchers are updated
```

One of the first steps we will cover in this module is searching for a proper `exploit` for our `target`. Nevertheless, we need to have a detailed perspective on the `target` itself before attempting any exploitation. This involves the `Enumeration` process, which precedes any type of exploitation attempt.

During `Enumeration`, we have to look at our target and identify which public-facing services are running on it. For example, is it an HTTP server? Is it an FTP server? Is it an SQL Database? These different `target` typologies vary substantially in the real world. We will need to start with a thorough `scan` of the target's IP address to determine what service is running and what version is installed for each service.

We will notice as we go along that versions are the key components during the `Enumeration` process that will allow us to determine if the target is vulnerable or not. Unpatched versions of previously vulnerable services or outdated code in a publicly accessible platform will often be our entry point into the `target` system.

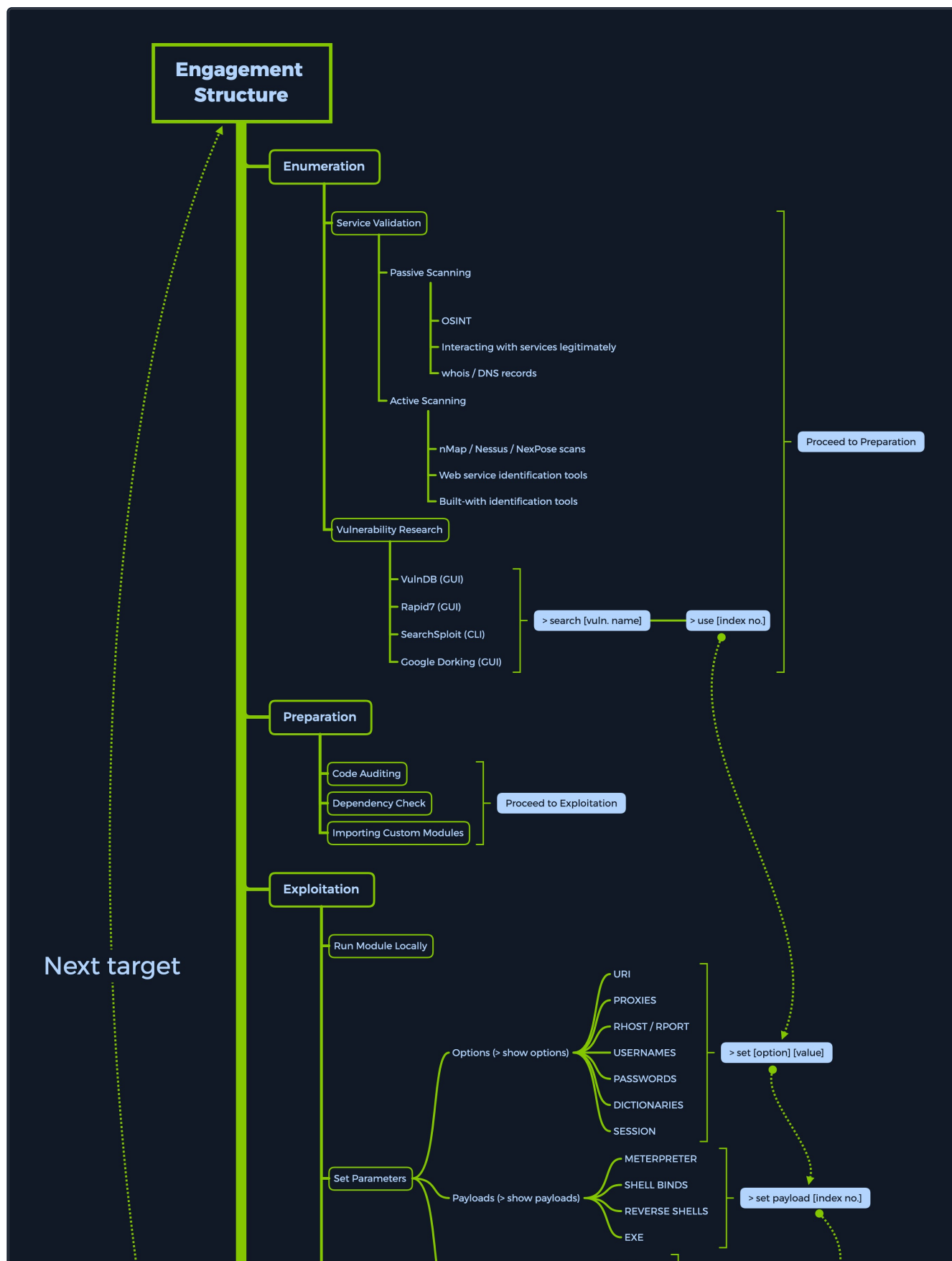
MSF Engagement Structure

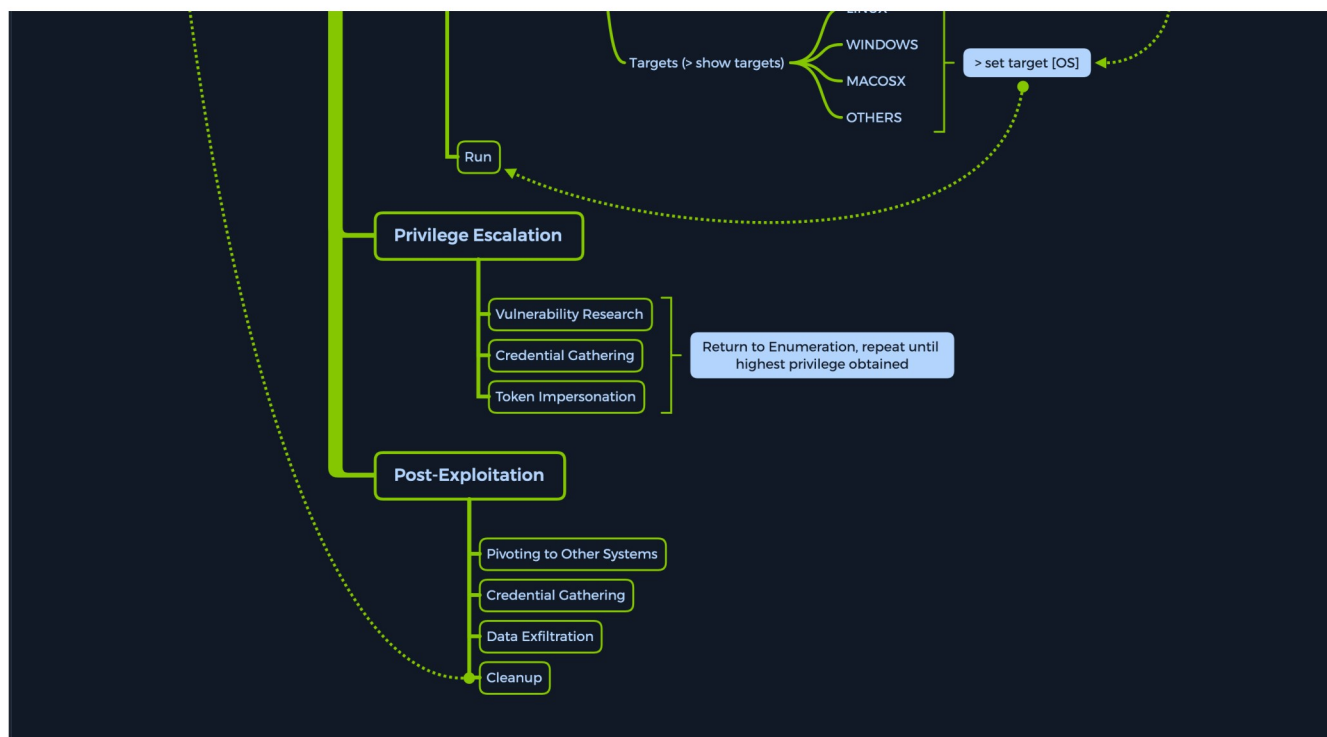
The MSF engagement structure can be divided into five main categories.

- Enumeration
- Preparation
- Exploitation
- Privilege Escalation
- Post-Exploitation

This division makes it easier for us to find and select the appropriate MSF features in a more structured way and to work with them accordingly. Each of these categories has different subcategories that are intended for specific purposes. These include, for example, Service Validation and Vulnerability Research.

It is therefore crucial that we familiarize ourselves with this structure. Therefore, we will look at this framework's components to better understand how they are related.





We will go through each of these categories during the module, but we recommend looking at the individual components ourselves and digging deeper. Experimenting with the different functions is an integral part of learning a new tool or skill. Therefore, we should try out everything imaginable here in the following labs and analyze the results independently.

[< Previous](#)[Next >](#)[✔ Mark Complete & Next](#)[📄 Cheat Sheet](#)

Table of Contents

Introduction

[Preface](#)[Introduction to Metasploit](#)[Introduction to MSFconsole](#)

MSF Components

[📦 Modules](#)[Targets](#)[📦 Payloads](#)[Encoders](#)[Databases](#)[Plugins & Mixins](#)

MSF Sessions

[📦 Sessions & Jobs](#)[📦 Meterpreter](#)

Additional Features

[Writing & Importing Modules](#)

Writing & importing modules

Introduction to MSFVenom

Firewall and IDS/IPS evasion

Metasploit-Framework Updates - August 2020

My Workstation

OFFLINE

▶ Start Instance

∞ / 1 spawns left